

OBJECT ORIENTED SYSTEMS (OOS) — 2023

QUESTION 3 COMPLETE DETAILED SOLUTION

Introduction

Question 3 of the 2023 Object Oriented Systems examination belongs to CO3 and focuses on some of the most advanced and conceptually rich features of Java programming such as:

-  Reflection API
-  Runtime Introspection
-  Generic Programming
-  Method Invocation
-  Dynamic Class Inspection
-  Generic Classes and Methods
-  Type Safety
-  Reusable Programming Structures

These concepts are extremely important because they form the foundation of enterprise frameworks, dependency injection systems, ORM frameworks, reusable software libraries, and advanced object-oriented architecture.

Modern Java technologies like Spring Framework, Hibernate, JUnit, and Java Collections Framework internally use reflection and generic programming extensively  .

This question therefore tests not only programming syntax but also deep conceptual understanding of runtime Java behavior.

QUESTION 3(a)

 **Explain Reflection in Java.**

 **Write suitable code snippets to:**

- **inspect methods of a class,**
- **invoke methods dynamically,**
- **and access private members using reflection.**

 **Answer**

One of the most powerful capabilities provided by Java is the concept of **Reflection**. Reflection is a mechanism through which a Java program can inspect and manipulate its own structure dynamically during runtime.

Normally in traditional programming, method calls, variable accesses, and object structures are determined during compilation. However, reflection allows:

- runtime class inspection,
- runtime method discovery,

- runtime field access,
- and dynamic invocation.

This capability makes Java highly flexible and extensible 🚀.

☀️ What is Reflection?

Reflection is a feature provided by the `java.lang.reflect` package. It allows a Java program to:

- obtain metadata about classes,
- inspect constructors,
- inspect methods,
- inspect fields,
- dynamically invoke methods,
- and even bypass access restrictions.

This process is also known as 🗨️ **Introspection** because the program “looks into itself” during execution.

☀️ Why Reflection is Needed?

Reflection is heavily used in:

- Spring Framework
- Hibernate
- Dependency Injection
- IDEs
- Testing frameworks
- Serialization systems

For example, Spring dynamically creates objects, injects dependencies, and invokes methods without explicit hardcoded object creation. Reflection makes this possible 🚀.

☀️ Important Reflection Classes

Class	Purpose
<code>Class</code>	Represents class metadata
<code>Method</code>	Represents methods
<code>Field</code>	Represents variables
<code>Constructor</code>	Represents constructors

☀️ Obtaining Class Object

Reflection begins by obtaining a `Class` object.

✅ Example

```
class Demo {  
}
```

```
class Main {  
    public static void main(String args[]) {  
        Demo d = new Demo();  
        Class c = d.getClass();  
        System.out.println(c.getName());  
    }  
}
```

Output

```
Demo
```

🌟 **Explanation** The method `getClass()` returns a metadata object representing the class structure.

🌟 Inspecting Methods of a Class

Reflection can inspect all methods dynamically.

✅ Program to Inspect Methods

```
import java.lang.reflect.*;  
  
class Sample {  
    void show() {  
    }  
  
    void display() {  
    }  
}  
  
class Main {  
    public static void main(String args[]) {  
        Class c = Sample.class;  
        Method methods[] = c.getDeclaredMethods();  
  
        for(Method m : methods) {  
            System.out.println(m.getName());  
        }  
    }  
}
```

Output

```
show  
display
```

🌟 **Deep Explanation** The method `getDeclaredMethods()` returns all methods declared inside the class. Each method is represented using a `Method` object. Thus runtime inspection becomes possible.

🌟 Dynamic Method Invocation

Reflection can invoke methods dynamically during runtime.

✅ Program for Dynamic Invocation

```
import java.lang.reflect.*;

class Test {
    void greet() {
        System.out.println("Hello Java");
    }
}

class Main {
    public static void main(String args[]) throws Exception {
        Test obj = new Test();
        Class c = obj.getClass();

        Method m = c.getMethod("greet");
        m.invoke(obj);
    }
}
```

🖥️ Output

```
Hello Java
```

🌟 Step-by-Step Execution

1. Initially, `getMethod("greet")` retrieves metadata of the `greet()` method.
2. Then, `m.invoke(obj)` executes the method dynamically during runtime.

Thus the actual method call is determined dynamically instead of compile-time invocation 🚀.

🌟 Accessing Private Members

Reflection can also bypass access restrictions.

✅ Program to Access Private Method

```
import java.lang.reflect.*;

class Secret {
    private void hidden() {
```

```
        System.out.println("Private Method");
    }
}

class Main {
    public static void main(String args[]) throws Exception {
        Secret obj = new Secret();
        Class c = obj.getClass();

        Method m = c.getDeclaredMethod("hidden");
        m.setAccessible(true);
        m.invoke(obj);
    }
}
```

Output

```
Private Method
```

💡 **Why `setAccessible(true)` is Needed?** Normally private members cannot be accessed outside the class. The statement `setAccessible(true)` disables Java access checking temporarily. Thus encapsulation restrictions are bypassed ⚠️.

💡 Advantages of Reflection

Reflection provides numerous advantages:

- Firstly, it enables highly dynamic software systems.
- Secondly, it supports framework development.
- Thirdly, it allows runtime inspection and dynamic invocation.
- Finally, it improves flexibility and extensibility significantly.

⚠️ Drawbacks of Reflection

Reflection also has disadvantages. It reduces performance, increases complexity, bypasses security restrictions, and may violate encapsulation. Thus reflection should be used carefully.

💡 JVM Perspective

When reflection is used, the JVM loads metadata, creates Method/Field objects, and dynamically resolves members during runtime. This introduces additional overhead compared to direct method invocation.

🎯 Conclusion

Reflection enables Java programs to inspect and manipulate class structures dynamically during runtime. It provides enormous flexibility and forms the backbone of modern enterprise Java frameworks 🚀.

QUESTION 3(b)

? **Explain Generic Programming in Java.**

? **Write suitable programs to demonstrate:**

- **generic class,**
- **generic method,**
- **and advantages of generics.**

✓ **Answer**

One of the major weaknesses of older Java versions was the lack of type safety in reusable collections and reusable methods. Before generics were introduced, programmers frequently used:

- `Object` references,
- explicit type casting,
- and unsafe data manipulation.

This created runtime errors, `ClassCastException`, and poor maintainability ⚠️.

To solve this problem, Java introduced: 🔥 **Generic Programming**

Generics allow classes, methods, and interfaces to operate on different datatypes while maintaining compile-time type safety.

💡 **What are Generics?**

Generics enable parameterized datatypes. Instead of writing separate classes for `Integer`, `String`, or `Double`, we write ONE reusable generic structure.

💡 **Syntax of Generics**

```
<T>
```

Here, `T` represents the type parameter.

💡 **Why Generics are Needed?**

Generics provide code reusability, type safety, compile-time checking, reduced casting, and cleaner programs.

💡 **Generic Class**

A generic class operates on parameterized datatypes.

✓ **Example of Generic Class**

```
class Box<T> {  
    T value;  
  
    void set(T value) {  
        this.value = value;  
    }  
}
```

```
T get() {
    return value;
}

class Main {
    public static void main(String args[]) {
        Box<Integer> b1 = new Box<>();
        b1.set(100);
        System.out.println(b1.get());

        Box<String> b2 = new Box<>();
        b2.set("Java");
        System.out.println(b2.get());
    }
}
```

Output

```
100
Java
```

💡 **Deep Explanation** The generic class `Box<T>` can operate on `Integer`, `String`, `Double`, or any datatype. Thus one reusable structure supports multiple types.

💡 Generic Method

A method can also become generic independently.

✅ Example of Generic Method

```
class Demo {
    <T> void show(T value) {
        System.out.println(value);
    }

    public static void main(String args[]) {
        Demo d = new Demo();
        d.show(100);
        d.show("Java");
        d.show(5.5);
    }
}
```

Output

```
100  
Java  
5.5
```

☀️ **Explanation** The method `<T> void show(T value)` accepts ANY datatype dynamically. The compiler automatically determines the datatype during compilation.

☀️ Type Safety

Before generics:

```
ArrayList list = new ArrayList();
```

This allowed mixed datatypes, which caused runtime errors.

With generics:

```
ArrayList<String> list = new ArrayList<>();
```

The compiler prevents invalid datatype insertion. Thus compile-time safety improves 🚀.

☀️ Advantages of Generics

Generics provide several important advantages:

- Firstly, they improve type safety.
- Secondly, they eliminate unnecessary casting.
- Thirdly, they improve code reusability.
- Finally, they make programs cleaner and easier to maintain.

☀️ Example of Compile-Time Safety

```
ArrayList<Integer> list = new ArrayList<>();  
list.add("Java"); // Compiler immediately generates an error.
```

Thus many runtime problems are prevented early.

☀️ Generic Constraints

Generics can also restrict datatypes using bounds:

```
<T Number extends>
```


This allows only numeric types to be passed as the parameter.

🔦 JVM Perspective

Internally Java uses **type erasure**. During compilation, generic information is removed and converted into **Object** references. The compiler inserts type checks and casts automatically.

⚠️ Limitations of Generics

Generics:

- cannot directly use primitive datatypes (must use wrapper classes like **Integer**),
- cannot instantiate type parameter objects directly (**new T()**),
- and do not exist fully during runtime due to type erasure.

🎯 Conclusion

Generic Programming enables reusable, type-safe, and maintainable software development in Java. It eliminates many runtime errors and forms a crucial part of modern Java programming 🚀.